Team notebook: Tyler / Hugo / Edwin

1. Index: constraints, keywords, traps · 2. Template · 3. Python standard library · 4. Greedy + techniques · 5. Data structures · 6. Math · 7. Geometry · 8. Graphs · 9. DP · 10. Strings

1. Index: constraints, keywords, traps (index.md)

Constraints -> what complexity you can afford

n	Allowed	Think
<= 10-12	O(n!)	permutations, brute force
<= 20-25	O(2^n * n)	bitmask DP, subsets, meet in the middle (n up to 40)
<= 400-500	O(n^3)	Floyd-Warshall, interval DP
<= 5,000	O(n^2)	2D DP, all pairs
<= 1e5-2e5	O(n log n)	sort, heap, segment tree, binary search, Dijkstra
<= 1e6	O(n) or O(n log n)	sieve, linear scan, two pointers
<= 1e9	O(sqrt n) or O(log n)	math, divisor loops
<= 1e18	O(log n) or formula	closed form, binary search on the answer, fast exponentiation

Python does roughly 1e7 simple steps per second. If it is too slow, switch to C++.

Phrase -> technique

The statement says	Try
shortest path, minimum cost to travel	BFS (unit), 0-1 BFS, Dijkstra (nonneg), Bellman-Ford (negative)
connected, merge groups, same component	DSU
order, before/after, dependencies, prerequisites	topological sort
minimum X such that ... is possible	binary search on the answer
number of ways, modulo 998244353	DP or combinatorics (factorials, inverses)
maximum number of non-overlapping	greedy by end time
minimum number of operations	BFS over states, greedy, or DP
k-th smallest / largest	sort, heap, binary search
range sum / range min with updates	Fenwick, segment tree, sparse table (no updates): data_structures.py
sum / min of every window of size k	prefix sums, monotonic deque: data_structures.py
next greater / smaller element	monotonic stack
substring, pattern, prefix	KMP, Z, hashing, trie
convex, farthest points, polygon area	cross product, hull, rotating calipers
every pair of vertices, tree	rerooting, LCA, centroid, counting by edges
grid, paths right/down	DP
game, both play optimally	Sprague-Grundy, minimax DP, parity
interactive	flush after each print, randomization
exactly one cycle (n vertices, n edges)	peel leaves, hanging trees
minimize sum of squares / convex cost	Cauchy-Schwarz, Lagrange, ternary search

Traps to check before submitting

- multiple test cases: reset everything per case
- overflow (C++ long long), modulo applied after every multiply
- 0-index vs 1-index
- scanning a list when you should use a set or dict
- recursion depth in Python (use an iterative stack or raise the limit)
- printing floats with a fixed number of decimals
- ordered vs unordered pairs, k = 0, n = 1 edge cases
- test the sample and one tiny edge case before submitting

2. Template (template.py)

```
import sys
from collections import defaultdict, Counter, deque
import heapq, bisect

input = sys.stdin.readline
sys.setrecursionlimit(1 << 20)
MOD = 998244353


def main():
    pass


main()
```

3. Python standard library (python_stdlib.md)

Docs: https://docs.python.org/3/library/ (collections, heapq, bisect, itertools, functools, math)

1. Input and output

```
import sys
input = sys.stdin.readline            # fast line reader; includes the trailing
newline
n = int(input())                      # one integer
a, b = map(int, input().split())      # several integers on one line
arr = list(map(int, input().split())) # a list of integers
s = input().strip()                   # a string without the newline

data = sys.stdin.buffer.read().split()   # alternative: read everything, walk with
an index

print(*arr)                           # numbers separated by spaces
print("\n".join(map(str, arr)))       # one per line, a single print call
print(f"{3.14159:.9f}")               # fixed decimals
```

2. Lists, sorting, tuples

```
nums = [5, 2, 9, 1]
nums.sort()                          # in place
nums.sort(reverse=True)
pairs = [(2, "b"), (1, "z"), (2, "a")]
pairs.sort(key=lambda p: (p[0], p[1]))       # sort by first, then second
words = sorted(["bb", "a", "ccc"], key=len)  # returns a new list
total, biggest = sum(nums), max(nums)
best = max(pairs, key=lambda p: p[1])        # max by a key
grid = [[0] * 4 for _ in range(3)]           # 3 rows x 4 columns (NOT [[0]*4]*3)
prefix = [0]
for x in nums:
    prefix.append(prefix[-1] + x)            # prefix[i] = sum of the first i
items
```

3. set and dict (hash tables, O(1) average)

```
seen = set()
seen.add(3); seen.discard(3)         # discard does not raise if missing
print(3 in seen)                     # O(1); never test membership on a list

d = {}
d["x"] = 1
d["x"] = d.get("x", 0) + 1           # default if missing
for key, value in d.items():
    pass
print(sorted(d))                     # sorted keys
```

4. collections

```
from collections import Counter, defaultdict, deque

c = Counter([1, 2, 2, 3, 3, 3])      # counts: c[3] == 3, c[99] == 0 (never
KeyError)
print(c.most_common(1))              # [(3, 3)]
for value, count in c.items():
    pass

adj = defaultdict(list)              # graph adjacency list
adj[1].append(2); adj[2].append(1)
freq = defaultdict(int)              # auto 0
freq["a"] += 1

q = deque([1, 2, 3])                 # fast queue (BFS)
q.append(4); first = q.popleft()     # O(1) at both ends
```

5. heapq (min-heap)

```
import heapq
h = []
heapq.heappush(h, (5, "a"))          # tuples compare by first item
heapq.heappush(h, (2, "b"))
smallest = heapq.heappop(h)          # (2, "b")
heapq.heapify(nums)                  # turn a list into a heap in O(n)
# max-heap trick: push negatives
mh = []                              # a separate heap (do not mix ints with
tuples)
heapq.heappush(mh, -7); heapq.heappush(mh, -3)
largest = -heapq.heappop(mh)         # 7
```

6. bisect (binary search on a SORTED list)

```
import bisect
a = [1, 3, 3, 5, 9]
bisect.bisect_left(a, 3)    # 1: first index with a[i] >= 3
bisect.bisect_right(a, 3)   # 3: first index with a[i] > 3
count_of_3 = bisect.bisect_right(a, 3) - bisect.bisect_left(a, 3)   # 2
bisect.insort(a, 4)         # insert and keep sorted (O(n) insert)
```

7. itertools and functools

```
from itertools import permutations, combinations, product, accumulate
from functools import lru_cache, cmp_to_key

list(permutations([1, 2, 3], 2))      # ordered choices of 2
list(combinations([1, 2, 3], 2))      # unordered choices of 2
list(product([0, 1], repeat=3))       # all 3-bit strings
list(accumulate([1, 2, 3, 4]))        # [1, 3, 6, 10]: prefix sums

@lru_cache(maxsize=None)              # memoization for recursive DP
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

# custom comparator: return negative, 0, positive
nums.sort(key=cmp_to_key(lambda x, y: (x > y) - (x < y)))
```

8. math and numbers

```
import math
math.gcd(12, 18)            # 6
math.isqrt(17)              # 4  (exact integer square root; never use int(x**0.5)
for big x)
math.factorial(5)           # 120
math.comb(5, 2)             # 10 (n choose k, exact)
math.log2(8), math.ceil(7 / 2), 7 // 2, 7 % 2, divmod(7, 2)   # use // for integer
division
pow(2, 10, 1_000_000_007)   # modular exponent: (2**10) % mod, fast
pow(3, -1, 11)              # modular inverse (Python 3.8+); for a prime p use
pow(x, p-2, p)
float("inf")                # infinity for min/max initial values
```

9. Strings

```
s = "hello world"
s.split()                   # ['hello', 'world']
" ".join(["a", "b"])        # 'a b'
s[::-1]                     # reverse
s.count("l"), s.find("o"), s.startswith("he")
ord("a"), chr(97)           # character <-> code
s.upper(), s.lower(), s.strip()
```

10. Safety settings and common mistakes

```
import sys
sys.setrecursionlimit(1 << 20)        # deep recursion; for n above ~1e5 prefer an
iterative stack
```

- `[[0]*m]*n` shares one row n times. Use `[[0]*m for _ in range(n)]`.
- Do not use `x in some_list` inside a loop. Use a set.
- `input()` returns a string, so convert with `int(...)`.
- Reading with `sys.stdin.readline` leaves a trailing newline; `.strip()` strings.
- `/` gives a float; `//` gives an integer.
- Sorting a list of tuples sorts by the first item, then the second, and so on.
- `list.pop(0)` and `list.insert(0, x)` are O(n). Use `deque`.
- Python has no built-in ordered set or sorted container. Use `bisect` on a sorted list, a heap, or switch to C++ `std::set`.

4. Greedy + techniques (techniques.py)

```
"""Greedy + common techniques page. Self-test: python3 techniques.py

GREEDY: guess a rule, then try to break it with a tiny counterexample before
coding.
  max non-overlapping intervals        -> sort by END, take if start >= last end
  min points/arrows hitting intervals  -> sort by END, shoot at end
  min rooms / max overlap              -> sweep: +1 at start, -1 at end, sorted
```

```
events
  schedule to minimize lateness        -> sort by deadline
  minimize sum of waiting times        -> shortest job first
  pair up to minimize max pair sum     -> sort, pair smallest with largest (two
pointers)
  merge cost (always join 2 smallest)  -> heap (Huffman)
  exchange argument: if swapping two adjacent items never helps, the sort key is
right
     e.g. sort by a/b  -> compare with cross-multiplication: key=cmp_to_key
TECHNIQUES
  many "add v to a[l..r]" then read all -> difference array
  coordinates up to 1e9 but n small     -> coordinate compression
  pairs/triples with sum target, sorted -> two pointers
  longest window with property          -> sliding window (grow right, shrink
left)
  n <= 40, subset sums                  -> meet in the middle
  count subarrays with sum k            -> prefix sum + Counter of seen prefixes
  minimize the maximum / maximize min   -> binary search on the answer
(math_nt.first_true)
"""
import heapq
from bisect import bisect_left, bisect_right
from collections import Counter
from functools import cmp_to_key


def max_non_overlapping(intervals):
    """[(s, e)] closed intervals; touching at an endpoint counts as overlapping"""
    cnt, last = 0, float('-inf')
    for s, e in sorted(intervals, key=lambda x: x[1]):
        if s > last:
            cnt += 1
            last = e
    return cnt


def max_overlap(intervals):
    """max intervals covering one point; [s, e) half-open. Ends sort before starts
at ties."""
    ev = []
    for s, e in intervals:
        ev.append((s, 1))
        ev.append((e, -1))
    ev.sort()
    cur = best = 0
    for _, d in ev:
        cur += d
        best = max(best, cur)
    return best


def huffman_cost(a):
    """repeatedly merge the two smallest; cost = sum of merged values"""
    h = list(a)
    heapq.heapify(h)
    cost = 0
    while len(h) > 1:
        x = heapq.heappop(h) + heapq.heappop(h)
        cost += x
        heapq.heappush(h, x)
    return cost


def diff_array(n, updates):
    """updates = [(l, r, v)] add v to a[l..r] inclusive. O(n + q)."""
    D = [0] * (n + 1)
    for l, r, v in updates:
        D[l] += v
        D[r + 1] -= v
    out, cur = [], 0
    for i in range(n):
        cur += D[i]
        out.append(cur)
    return out


def compress(a):
    """values -> 0..k-1 keeping order. Returns (ranks, sorted unique values)."""
    vals = sorted(set(a))
    idx = {v: i for i, v in enumerate(vals)}
    return [idx[x] for x in a], vals


def two_sum_sorted(a, target):
    """a sorted. returns indices (i, j), i < j, with a[i] + a[j] == target, or
None"""
    i, j = 0, len(a) - 1
    while i < j:
        s = a[i] + a[j]
        if s == target:
            return i, j
        if s < target:
            i += 1
        else:
            j -= 1
    return None


def longest_window_at_most_k_distinct(a, k):
    """sliding window template: grow right, shrink left while invalid"""
    cnt, left, best = Counter(), 0, 0
    for right, x in enumerate(a):
        cnt[x] += 1
        while len(cnt) > k:
            cnt[a[left]] -= 1
            if cnt[a[left]] == 0:
                del cnt[a[left]]
            left += 1
        best = max(best, right - left + 1)
    return best


def count_subarrays_sum(a, k):
    """number of (l, r) with sum(a[l..r]) == k. Works with negatives."""
    seen, pre, total = Counter({0: 1}), 0, 0
    for x in a:
        pre += x
        total += seen[pre - k]
        seen[pre] += 1
    return total


def subset_sums(a):
    sums = [0]
    for x in a:
        sums += [s + x for s in sums]
    return sums


def meet_in_middle_count(a, target):
```

```
    """number of subsets with sum == target, n <= ~40. O(2^(n/2) log)"""
    L = subset_sums(a[:len(a) // 2])
    R = sorted(subset_sums(a[len(a) // 2:]))
    return sum(bisect_right(R, target - s) - bisect_left(R, target - s) for s in
L)


def sort_by_ratio(items):
    """items (a, b) with b > 0, sort by a/b ascending WITHOUT floats"""
    return sorted(items, key=cmp_to_key(lambda x, y: x[0] * y[1] - y[0] * x[1]))


# bit tricks: x & -x lowest bit | x & (x-1) drop lowest bit | bin(x).count('1')
popcount
# iterate submasks of m: s = m; while s: ...; s = (s - 1) & m
# x >> i & 1  is bit i      | 1 << n  is 2^n          | x.bit_length()
```

5. Data structures (data_structures.py)

```
"""Data structures page. Every block is tested by the brute-force check at the
bottom
(run: python3 data_structures.py).

WHICH ONE?
  range SUM, array never changes      -> prefix sums          O(1) query
  range SUM, with updates             -> Fenwick              O(log n) both
  range MIN/MAX, array never changes  -> sparse table         O(1) query
  range MIN/MAX/SUM, with updates     -> segment tree         O(log n) both
  min/max of every window of size k   -> monotonic deque      O(n) total
  sum of a rectangle in a grid        -> 2D prefix sums       O(1) query

Neutral fill values: sum -> 0, min -> inf, max -> -inf.
Test edges: (0,0), (0,n-1), (n-1,n-1), and every (l,r) on a small array vs brute
force.
"""
from collections import deque

INF = float('inf')


# ---------- 1D prefix sums ----------
# P[k] = a[0] + ... + a[k-1];  sum(a[l..r]) = P[r+1] - P[l]   (0-indexed,
inclusive)
def prefix_sums(a):
    P = [0] * (len(a) + 1)
    for i, x in enumerate(a):
        P[i + 1] = P[i] + x
    return P


# ---------- 2D prefix sums ----------
# S[i][j] = sum of g[0..i-1][0..j-1]
# rect(r1,c1,r2,c2) inclusive = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
[c1]
def prefix_2d(g):
    n, m = len(g), len(g[0])
    S = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n):
        for j in range(m):
            S[i + 1][j + 1] = g[i][j] + S[i][j + 1] + S[i + 1][j] - S[i][j]
    return S


def rect(S, r1, c1, r2, c2):
    return S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1] + S[r1][c1]


# ---------- Fenwick tree (BIT): point add, prefix sum ----------
# 1-INDEXED. T[i] holds the sum of a block ending at i, length = lowest bit of i.
# "set a[k] = u"  ->  add(k, u - a[k]); a[k] = u      (keep the plain array a!)
# range sum a[l..r] = prefix(r) - prefix(l - 1)
# Does NOT do range min (min can't be subtracted) -> use segment tree.
class Fenwick:
    def __init__(self, n):
        self.n = n
        self.T = [0] * (n + 1)

    def add(self, i, d):            # a[i] += d
        T, n = self.T, self.n
        while i <= n:
            T[i] += d
            i += i & -i             # climb to every box that contains i

    def prefix(self, i):            # a[1] + ... + a[i]
        T, s = self.T, 0
        while i > 0:
            s += T[i]
            i -= i & -i             # hop down the staircase
        return s

    def range_sum(self, l, r):      # a[l..r], 1-indexed inclusive
        return self.prefix(r) - self.prefix(l - 1)


# ---------- Sparse table: static range min, O(1) query ----------
# Level i has exactly n - 2^i + 1 blocks (no filler: a bad index crashes loudly).
# Query: two blocks of length 2^k that overlap -- fine for min/max/gcd, WRONG for
sum.
def sparse_build(a, f=min):
    n = len(a)
    sp = [a[:]]
    i = 1
    while (1 << i) <= n:
        prev, half = sp[i - 1], 1 << (i - 1)
        sp.append([f(prev[j], prev[j + half]) for j in range(n - (1 << i) + 1)])
        i += 1
    return sp


def sparse_query(sp, l, r, f=min):  # 0-indexed inclusive
    k = (r - l + 1).bit_length() - 1
    return f(sp[k][l], sp[k][r - (1 << k) + 1])


# ---------- Segment tree: point set, range query (any associative op) ----------
# 0-indexed, query is HALF-OPEN [l, r).  Iterative, bottom-up.
# Change op/neutral: min/INF, max/-INF, sum/0, gcd/0.
class SegTree:
    def __init__(self, a, op=min, neutral=INF):
        n = len(a)
        self.n, self.op, self.e = n, op, neutral
        self.t = [neutral] * n + list(a)
        for i in range(n - 1, 0, -1):
            self.t[i] = op(self.t[2 * i], self.t[2 * i + 1])

    def set(self, i, v):            # a[i] = v
        t, op = self.t, self.op
        i += self.n
        t[i] = v
        i >>= 1
```

```
        while i:
            t[i] = op(t[2 * i], t[2 * i + 1])
            i >>= 1

    def query(self, l, r):          # op over a[l..r-1]
        t, op = self.t, self.op
        resl = resr = self.e
        l += self.n
        r += self.n
        while l < r:
            if l & 1:
                resl = op(resl, t[l]); l += 1
            if r & 1:
                r -= 1; resr = op(t[r], resr)
            l >>= 1
            r >>= 1
        return op(resl, resr)


# ---------- Monotonic deque: min of every window of size k ----------
# For max: flip the comparison to <=.
def window_mins(a, k):
    dq, out = deque(), []
    for i, x in enumerate(a):
        while dq and a[dq[-1]] >= x:    # bigger ones behind x can never be the
min
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:              # front fell out of the window
            dq.popleft()
        if i >= k - 1:
            out.append(a[dq[0]])
    return out


# ---------- Monotonic stack: next smaller element to the right ----------
# res[i] = index of first j > i with a[j] < a[i], or -1.  (next greater: flip to
<)
def next_smaller(a):
    res, st = [-1] * len(a), []
    for i, x in enumerate(a):
        while st and a[st[-1]] > x:
            res[st.pop()] = i
        st.append(i)
    return res
```

6. Math (math_nt.py)

```
"""Math page: number theory, combinatorics, binary search. Self-test: python3
math_nt.py

SPOT IT
  "modulo 1e9+7 / 998244353"   -> work mod p the whole time, divide = multiply by
inverse
  "number of ways to choose"   -> nCr with factorial tables
  "gcd/lcm/divisible"          -> gcd, sieve, factorize
  "a^b with huge b"            -> pow(a, b, m)   (built in, fast)
  "minimum X such that ok(X)"  -> binary search on the answer (ok must be
monotone)
  "a*x + b*y = c"              -> extended gcd
  "x = r1 mod m1, x = r2 mod m2" -> CRT
Python ints never overflow; math.isqrt(n) is the exact integer sqrt;
math.comb(n,k) exact.
"""
from math import gcd, isqrt

MOD = 10**9 + 7


def lcm(a, b):
    return a // gcd(a, b) * b


def ext_gcd(a, b):
    """returns (g, x, y) with a*x + b*y = g = gcd(a, b)"""
    x0, y0, x1, y1 = 1, 0, 0, 1
    while b:
        q = a // b
        a, b = b, a - q * b
        x0, x1 = x1, x0 - q * x1
        y0, y1 = y1, y0 - q * y1
    return a, x0, y0


def modinv(a, m=MOD):
    """m prime: pow(a, m-2, m).  General m (gcd(a,m)=1): pow(a, -1, m) in Python
3.8+"""
    return pow(a, -1, m)


def crt(r1, m1, r2, m2):
    """x = r1 (mod m1), x = r2 (mod m2). Returns (x, lcm) or None if
impossible."""
    g, p, _ = ext_gcd(m1, m2)
    if (r2 - r1) % g:
        return None
    l = m1 // g * m2
    x = (r1 + (r2 - r1) // g * p % (m2 // g) * m1) % l
    return x, l


def sieve(n):
    """is_p[i] for 0..n, and list of primes. O(n log log n). n up to ~1e7 in
PyPy."""
    is_p = bytearray([1]) * (n + 1)
    is_p[0:2] = b'\x00\x00'[:min(2, n + 1)]
    for i in range(2, isqrt(n) + 1):
        if is_p[i]:
            is_p[i * i::i] = bytearray(len(range(i * i, n + 1, i)))
    return is_p, [i for i in range(n + 1) if is_p[i]]


def spf_sieve(n):
    """smallest prime factor for 0..n -> factorize any x <= n in O(log x)"""
    spf = list(range(n + 1))
    for i in range(2, isqrt(n) + 1):
        if spf[i] == i:
            for j in range(i * i, n + 1, i):
                if spf[j] == j:
                    spf[j] = i
    return spf


def factorize(n):
    """trial division, O(sqrt n). Returns {prime: exponent}"""
    f, d = {}, 2
    while d * d <= n:
        while n % d == 0:
            f[d] = f.get(d, 0) + 1
            n //= d
```

```
        d += 1
    if n > 1:
        f[n] = f.get(n, 0) + 1
    return f


def divisors(n):
    small, large, d = [], [], 1
    while d * d <= n:
        if n % d == 0:
            small.append(d)
            if d != n // d:
                large.append(n // d)
        d += 1
    return small + large[::-1]


def phi(n):
    """Euler totient: count of 1..n coprime to n"""
    r = n
    for p in factorize(n):
        r -= r // p
    return r


class Comb:
    """nCr mod p (p prime, n < p) in O(1) after O(N) precompute."""
    def __init__(self, N, p=MOD):
        self.p = p
        f = [1] * (N + 1)
        for i in range(1, N + 1):
            f[i] = f[i - 1] * i % p
        inv = [1] * (N + 1)
        inv[N] = pow(f[N], p - 2, p)
        for i in range(N, 0, -1):
            inv[i - 1] = inv[i] * i % p
        self.f, self.inv = f, inv

    def C(self, n, k):
        if k < 0 or k > n:
            return 0
        return self.f[n] * self.inv[k] % self.p * self.inv[n - k] % self.p

    def P(self, n, k):          # ordered selections
        if k < 0 or k > n:
            return 0
        return self.f[n] * self.inv[n - k] % self.p
# stars and bars: ways to split n identical items into k groups (>=0 each) =
C(n+k-1, k-1)
# Catalan(n) = C(2n, n) / (n+1)   (balanced parens, binary trees, non-crossing)
# inclusion-exclusion: |A or B| = |A| + |B| - |A and B|; over subsets with sign
(-1)^|S|


def mat_mul(A, B, m=MOD):
    n, k, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(n)]
    for i in range(n):
        Ai, Ci = A[i], C[i]
        for t in range(k):
            a = Ai[t]
            if a:
                Bt = B[t]
                for j in range(p):
                    Ci[j] += a * Bt[j]
                    Ci[j] %= m
    return C


def mat_pow(A, e, m=MOD):
    """linear recurrences in O(k^3 log e): fib(n) = mat_pow([[1,1],[1,0]], n)[0]
[1]"""
    n = len(A)
    R = [[int(i == j) for j in range(n)] for i in range(n)]
    while e:
        if e & 1:
            R = mat_mul(R, A, m)
        A = mat_mul(A, A, m)
        e >>= 1
    return R


# ---------- binary search ----------
def first_true(lo, hi, ok):
    """smallest x in [lo, hi] with ok(x) True; ok must be F..F T..T. Returns hi+1
if none."""
    hi += 1
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo


def last_true(lo, hi, ok):
    """largest x in [lo, hi] with ok(x) True; ok must be T..T F..F. Returns lo-1
if none."""
    lo -= 1
    while lo < hi:
        mid = (lo + hi + 1) // 2
        if ok(mid):
            lo = mid
        else:
            hi = mid - 1
    return lo


def bsearch_real(lo, hi, ok, iters=100):
    """real-valued: smallest x with ok(x). 100 iterations is always enough."""
    for _ in range(iters):
        mid = (lo + hi) / 2
        if ok(mid):
            hi = mid
        else:
            lo = mid
    return hi


def ternary_max(lo, hi, f, iters=200):
    """max of a unimodal (up then down) real function on [lo, hi]"""
    for _ in range(iters):
        m1 = lo + (hi - lo) / 3
        m2 = hi - (hi - lo) / 3
        if f(m1) < f(m2):
            lo = m1
        else:
```

```
            hi = m2
    return (lo + hi) / 2
```

7. Geometry (geometry.py)

```
"""Geometry page. Points are tuples (x, y). Self-test: python3 geometry.py

RULES
  * Integer input -> stay in integers (cross products, squared distances). No
floats, no sqrt.
  * Floats only when forced (circles, angles); compare with EPS = 1e-9.
  * cross(o, a, b) > 0: o->a->b turns LEFT (counter-clockwise); < 0 right; = 0
collinear.
  * Polygon area = |shoelace| / 2; 2*area is an integer for integer points.
  * Pick's theorem: Area = I + B/2 - 1  (I interior lattice pts, B boundary
lattice pts)
  * boundary lattice points on segment = gcd(|dx|, |dy|)  (excluding one endpoint)
  * angle of a vector: math.atan2(y, x) in (-pi, pi];  math.hypot(dx, dy) = length
"""
from math import gcd, hypot, sqrt, atan2

EPS = 1e-9


def sub(a, b): return (a[0] - b[0], a[1] - b[1])
def dot(a, b): return a[0] * b[0] + a[1] * b[1]
def cross2(a, b): return a[0] * b[1] - a[1] * b[0]
def cross(o, a, b): return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] -
o[0])
def dist2(a, b): return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2


def sign(x):
    return (x > EPS) - (x < -EPS)


def on_segment(p, a, b):
    """p on closed segment ab (exact for ints)"""
    return cross(a, b, p) == 0 and min(a[0], b[0]) <= p[0] <= max(a[0], b[0]) \
        and min(a[1], b[1]) <= p[1] <= max(a[1], b[1])


def seg_intersect(a, b, c, d):
    """do closed segments ab and cd touch/cross? (exact for ints)"""
    d1, d2 = cross(a, b, c), cross(a, b, d)
    d3, d4 = cross(c, d, a), cross(c, d, b)
    if ((d1 > 0) != (d2 > 0)) and d1 != 0 and d2 != 0 and \
       ((d3 > 0) != (d4 > 0)) and d3 != 0 and d4 != 0:
        return True
    return on_segment(c, a, b) or on_segment(d, a, b) or on_segment(a, c, d) or
on_segment(b, c, d)


def line_intersection(a, b, c, d):
    """intersection point of infinite lines ab and cd (floats), None if
parallel"""
    r, s = sub(b, a), sub(d, c)
    den = cross2(r, s)
    if den == 0:
        return None
    t = cross2(sub(c, a), s) / den
    return (a[0] + t * r[0], a[1] + t * r[1])


def point_seg_dist(p, a, b):
    ab, ap = sub(b, a), sub(p, a)
    L = dot(ab, ab)
    if L == 0:
        return sqrt(dist2(p, a))
    t = max(0.0, min(1.0, dot(ap, ab) / L))
    return hypot(a[0] + t * ab[0] - p[0], a[1] + t * ab[1] - p[1])


def area2(poly):
    """twice the signed area (positive if counter-clockwise). Shoelace."""
    s = 0
    for i in range(len(poly)):
        x1, y1 = poly[i]
        x2, y2 = poly[(i + 1) % len(poly)]
        s += x1 * y2 - x2 * y1
    return s


def point_in_polygon(p, poly):
    """1 inside, 0 on boundary, -1 outside. Any simple polygon. Exact for ints."""
    n, inside = len(poly), False
    for i in range(n):
        a, b = poly[i], poly[(i + 1) % n]
        if on_segment(p, a, b):
            return 0
        if (a[1] > p[1]) != (b[1] > p[1]):
            # x-coordinate of crossing compared without division
            c = cross(a, b, p)
            if (c > 0) == (b[1] > a[1]):
                inside = not inside
    return 1 if inside else -1


def convex_hull(pts):
    """Andrew's monotone chain. Returns hull counter-clockwise, no collinear
points.
    To KEEP collinear boundary points change both <= 0 to < 0 (careful with all-
collinear)."""
    pts = sorted(set(pts))
    if len(pts) <= 2:
        return pts
    lower, upper = [], []
    for p in pts:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0:
            lower.pop()
        lower.append(p)
    for p in reversed(pts):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0:
            upper.pop()
        upper.append(p)
    return lower[:-1] + upper[:-1]


def diameter2(pts):
    """max squared distance between any two points: rotating calipers on the
hull"""
    h = convex_hull(pts)
    n = len(h)
    if n == 1:
        return 0
    if n == 2:
        return dist2(h[0], h[1])
    best, j = 0, 1
    for i in range(n):
        ni = (i + 1) % n
```

```
        while abs(cross(h[i], h[ni], h[(j + 1) % n])) > abs(cross(h[i], h[ni],
h[j])):
            j = (j + 1) % n
        best = max(best, dist2(h[i], h[j]), dist2(h[ni], h[j]))
    return best


def lattice_boundary(poly):
    return sum(gcd(abs(poly[i][0] - poly[i - 1][0]), abs(poly[i][1] - poly[i - 1]
[1]))
               for i in range(len(poly)))


def lattice_interior(poly):
    """Pick: I = A - B/2 + 1  ->  2I = 2A - B + 2"""
    return (abs(area2(poly)) - lattice_boundary(poly) + 2) // 2


def circle_line(c, r, a, b):
    """intersections of circle (center c, radius r) with line ab, as float
points"""
    d = sub(b, a)
    f = sub(a, c)
    A, B, C = dot(d, d), 2 * dot(f, d), dot(f, f) - r * r
    disc = B * B - 4 * A * C
    if disc < -EPS:
        return []
    disc = sqrt(max(0.0, disc))
    ts = {(-B - disc) / (2 * A), (-B + disc) / (2 * A)}
    return [(a[0] + t * d[0], a[1] + t * d[1]) for t in sorted(ts)]


def closest_pair2(pts):
    """min squared distance, O(n log n) sweep. pts: list of tuples (len >= 2)."""
    import bisect
    pts = sorted(pts)
    best = dist2(pts[0], pts[1])
    active, left = [], 0          # active sorted by (y, x)
    for p in pts:
        while (p[0] - pts[left][0]) ** 2 > best:
            q = pts[left]
            active.pop(bisect.bisect_left(active, (q[1], q[0])))
            left += 1
        d = int(sqrt(best)) + 1
        lo = bisect.bisect_left(active, (p[1] - d, -10**18))
        hi = bisect.bisect_right(active, (p[1] + d, 10**18))
        for y, x in active[lo:hi]:
            best = min(best, dist2(p, (x, y)))
        bisect.insort(active, (p[1], p[0]))
    return best
```

8. Graphs (graphs.py)

```
"""Graphs page. Nodes 0..n-1 (subtract 1 when input is 1-indexed!). Self-test:
python3 graphs.py

SPOT IT
  shortest path, all edges weight 1 / grid moves -> BFS
  weights 0 or 1                                 -> 0-1 BFS (deque)
  weights >= 0                                   -> Dijkstra
  negative weights / detect negative cycle       -> Bellman-Ford
  all pairs, n <= 400ish                         -> Floyd-Warshall
  connect everything cheapest                    -> Kruskal (sort edges + DSU)
  groups / connected? / merge online             -> DSU
  order with prerequisites / is there a cycle    -> topological sort
  two colors, no conflict, odd cycle             -> bipartite check
  mutual reachability, 2-SAT                     -> SCC (Kosaraju)
  tree: distance / ancestor queries              -> LCA binary lifting
  tree: farthest pair                            -> diameter (BFS twice)
  max flow / min cut / disjoint paths            -> Dinic
  match workers to jobs                          -> bipartite matching (Kuhn) or
Dinic
Python: NO recursive DFS on big graphs (use a stack). Build adj as list of lists.
Reading: n, m then m lines u v (w)  -> adj[u-1].append((v-1, w)); undirected: both
ways.
"""
import heapq
from collections import deque

INF = float('inf')


def dijkstra(adj, src, n):
    """adj[u] = list of (v, w), w >= 0. Returns dist list, inf if unreachable."""
    dist = [INF] * n
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue                       # stale entry
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(pq, (nd, v))
    return dist


def dijkstra_path(adj, src, dst, n):
    """also returns the path src..dst (or None)"""
    dist, par = [INF] * n, [-1] * n
    dist[src] = 0
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v], par[v] = d + w, u
                heapq.heappush(pq, (d + w, v))
    if dist[dst] == INF:
        return INF, None
    path, v = [], dst
    while v != -1:
        path.append(v)
        v = par[v]
    return dist[dst], path[::-1]


def bfs(adj, src, n):
    """unweighted shortest paths; -1 = unreachable"""
    dist = [-1] * n
    dist[src] = 0
    q = deque([src])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if dist[v] < 0:
```

```
                dist[v] = dist[u] + 1
                q.append(v)
    return dist


def grid_bfs(grid, sr, sc):
    """grid of strings, '#' = wall. Returns dist grid (-1 unreachable)."""
    R, C = len(grid), len(grid[0])
    dist = [[-1] * C for _ in range(R)]
    dist[sr][sc] = 0
    q = deque([(sr, sc)])
    while q:
        r, c = q.popleft()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] != '#' and dist[nr]
[nc] < 0:
                dist[nr][nc] = dist[r][c] + 1
                q.append((nr, nc))
    return dist


def bfs01(adj, src, n):
    """weights 0/1: 0-edge -> appendleft, 1-edge -> append"""
    dist = [INF] * n
    dist[src] = 0
    dq = deque([src])
    while dq:
        u = dq.popleft()
        for v, w in adj[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                (dq.appendleft if w == 0 else dq.append)(v)
    return dist


def bellman_ford(n, edges, src):
    """edges = [(u, v, w)]. Returns dist, or None if a negative cycle is reachable
from src."""
    dist = [INF] * n
    dist[src] = 0
    for _ in range(n - 1):
        changed = False
        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    for u, v, w in edges:
        if dist[u] != INF and dist[u] + w < dist[v]:
            return None
    return dist


def floyd(n, edges):
    """all pairs. d[i][i] < 0 after => i on a negative cycle. O(n^3)."""
    d = [[INF] * n for _ in range(n)]
    for i in range(n):
        d[i][i] = 0
    for u, v, w in edges:
        d[u][v] = min(d[u][v], w)
    for k in range(n):
        dk = d[k]
        for i in range(n):
            dik, di = d[i][k], d[i]
            if dik == INF:
                continue
            for j in range(n):
                if dik + dk[j] < di[j]:
                    di[j] = dik + dk[j]
    return d


class DSU:
    def __init__(self, n):
        self.p = list(range(n))
        self.sz = [1] * n

    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]
            x = self.p[x]
        return x

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return False
        if self.sz[a] < self.sz[b]:
            a, b = b, a
        self.p[b] = a
        self.sz[a] += self.sz[b]
        return True


def kruskal(n, edges):
    """edges = [(w, u, v)]. Returns (total weight, #edges used). #edges < n-1 =>
not connected."""
    d, total, used = DSU(n), 0, 0
    for w, u, v in sorted(edges):
        if d.union(u, v):
            total += w
            used += 1
    return total, used


def topo_sort(adj, n):
    """Kahn's algorithm. Returns order, or None if there is a cycle."""
    indeg = [0] * n
    for u in range(n):
        for v in adj[u]:
            indeg[v] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order if len(order) == n else None


def components(adj, n):
    """comp[v] = component id, iterative DFS"""
    comp, c = [-1] * n, 0
    for s in range(n):
        if comp[s] >= 0:
```

```
            continue
        comp[s] = c
        st = [s]
        while st:
            u = st.pop()
            for v in adj[u]:
                if comp[v] < 0:
                    comp[v] = c
                    st.append(v)
        c += 1
    return comp, c


def bipartite(adj, n):
    """returns color list (0/1) or None if an odd cycle exists"""
    col = [-1] * n
    for s in range(n):
        if col[s] >= 0:
            continue
        col[s] = 0
        q = deque([s])
        while q:
            u = q.popleft()
            for v in adj[u]:
                if col[v] < 0:
                    col[v] = col[u] ^ 1
                    q.append(v)
                elif col[v] == col[u]:
                    return None
    return col


def scc(adj, n):
    """Kosaraju, iterative. comp[v] in topological order of the condensation."""
    order, seen = [], [False] * n
    for s in range(n):
        if seen[s]:
            continue
        seen[s] = True
        st = [(s, 0)]
        while st:
            u, i = st.pop()
            if i < len(adj[u]):
                st.append((u, i + 1))
                v = adj[u][i]
                if not seen[v]:
                    seen[v] = True
                    st.append((v, 0))
            else:
                order.append(u)
    radj = [[] for _ in range(n)]
    for u in range(n):
        for v in adj[u]:
            radj[v].append(u)
    comp, c = [-1] * n, 0
    for s in reversed(order):
        if comp[s] >= 0:
            continue
        comp[s] = c
        st = [s]
        while st:
            u = st.pop()
            for v in radj[u]:
                if comp[v] < 0:
                    comp[v] = c
                    st.append(v)
        c += 1
    return comp, c
# 2-SAT: var x -> nodes 2x (true), 2x+1 (false). clause (a or b): edges ~a->b, ~b->a.
# unsatisfiable iff comp[2x] == comp[2x+1]; x = True iff comp[2x] > comp[2x+1].


class LCA:
    """tree with root; binary lifting. O(n log n) build, O(log n) query."""
    def __init__(self, adj, root=0):
        n = len(adj)
        self.LOG = LOG = max(1, n.bit_length())
        self.depth = depth = [0] * n
        up = [[root] * n for _ in range(LOG)]
        seen = [False] * n
        seen[root] = True
        st = [root]
        while st:
            u = st.pop()
            for v in adj[u]:
                if not seen[v]:
                    seen[v] = True
                    depth[v] = depth[u] + 1
                    up[0][v] = u
                    st.append(v)
        for k in range(1, LOG):
            prev, cur = up[k - 1], up[k]
            for v in range(n):
                cur[v] = prev[prev[v]]
        self.up = up

    def kth_ancestor(self, v, k):
        for i in range(self.LOG):
            if k >> i & 1:
                v = self.up[i][v]
        return v

    def lca(self, a, b):
        if self.depth[a] < self.depth[b]:
            a, b = b, a
        a = self.kth_ancestor(a, self.depth[a] - self.depth[b])
        if a == b:
            return a
        for k in range(self.LOG - 1, -1, -1):
            if self.up[k][a] != self.up[k][b]:
                a, b = self.up[k][a], self.up[k][b]
        return self.up[0][a]

    def dist(self, a, b):
        return self.depth[a] + self.depth[b] - 2 * self.depth[self.lca(a, b)]


def tree_diameter(adj, n):
    """BFS from any node -> farthest u; BFS from u -> farthest distance"""
    d = bfs(adj, 0, n)
    u = d.index(max(d))
    d2 = bfs(adj, u, n)
    return max(d2)


class Dinic:
    """max flow. add_edge(u, v, cap). O(V^2 E), fast in practice. Min cut = max flow."""
    def __init__(self, n):
```

```
        self.n = n
        self.g = [[] for _ in range(n)]      # edge = [to, cap, index of reverse]

    def add_edge(self, u, v, cap):
        self.g[u].append([v, cap, len(self.g[v])])
        self.g[v].append([u, 0, len(self.g[u]) - 1])

    def max_flow(self, s, t):
        flow, g = 0, self.g
        while True:
            level = [-1] * self.n
            level[s] = 0
            q = deque([s])
            while q:
                u = q.popleft()
                for v, cap, _ in g[u]:
                    if cap > 0 and level[v] < 0:
                        level[v] = level[u] + 1
                        q.append(v)
            if level[t] < 0:
                return flow
            it = [0] * self.n
            while True:                          # iterative DFS for blocking flow
                stack, path = [s], []
                found = False
                while stack:
                    u = stack[-1]
                    if u == t:
                        found = True
                        break
                    advanced = False
                    while it[u] < len(g[u]):
                        e = g[u][it[u]]
                        if e[1] > 0 and level[e[0]] == level[u] + 1:
                            stack.append(e[0])
                            path.append(e)
                            advanced = True
                            break
                        it[u] += 1
                    if not advanced:
                        stack.pop()
                        if path:
                            path.pop()
                            it[stack[-1]] += 1
                        level[u] = -1
                if not found:
                    break
                f = min(e[1] for e in path)
                for e in path:
                    e[1] -= f
                    g[e[0]][e[2]][1] += f
                flow += f


def bipartite_matching(adj, nL, nR):
    """Kuhn's algorithm. adj[l] = list of right nodes. Returns (size, matchR)."""
    matchR = [-1] * nR
    size = 0
    for s in range(nL):
        used = [False] * nR
        # iterative augmenting path search
        stack = [(s, 0)]
        parent = {}
        found = -1
        while stack and found < 0:
            l, i = stack.pop()
            while i < len(adj[l]):
                r = adj[l][i]
                i += 1
                if used[r]:
                    continue
                used[r] = True
                parent[r] = l
                if matchR[r] < 0:
                    found = r
                    break
                stack.append((l, i))
                stack.append((matchR[r], 0))
                break
        if found >= 0:
            r = found
            while r != -1:
                l = parent[r]
                prev = next((rr for rr in adj[l] if matchR[rr] == l), -1)
                matchR[r] = l
                r = prev
            size += 1
    return size, matchR
```

9. DP (dp.py)

```
"""DP page. Self-test: python3 dp.py

HOW TO WRITE ANY DP (do these 4 in a comment before coding)
  1. STATE:  dp[i][j] = <one sentence, in words>  (what do I need to know to
finish?)
  2. CHOICE: what is the last decision?  -> transition
  3. BASE:   smallest cases
  4. ORDER/ANSWER: fill so dependencies come first; which cell is the answer?
Size check: (#states) * (#choices) <= ~1e7 in PyPy.
"number of ways" -> add, mod.   "min/max" -> min/max.   "possible?" -> or.

SPOT IT
  pick items, capacity W           -> knapsack (0/1: loop w DOWN; unbounded: loop
w UP)
  count ways to make sum           -> coin change (order matters? loop sum
outside)
  longest increasing subsequence   -> LIS with bisect, O(n log n)
  two strings, match/edit          -> LCS / edit distance, dp[i][j] on prefixes
  grid, move right/down            -> dp[r][c] from top and left
  n <= 20, subsets / visit all     -> bitmask DP dp[mask][last]
  merge/split a range, l..r        -> interval DP dp[l][r], loop by length
  count numbers <= N with property -> digit DP (tight flag)
  tree, choose nodes               -> dp[v][taken?] from children (iterative post-
order)
"""
from bisect import bisect_left, bisect_right
from functools import lru_cache

MOD = 10**9 + 7
INF = float('inf')


def knapsack01(items, W):
    """items = [(weight, value)], each used at most once. max value with weight <=
W"""
    dp = [0] * (W + 1)
    for w, v in items:
        for c in range(W, w - 1, -1):          # DOWN: each item once
            if dp[c - w] + v > dp[c]:
```

```
                dp[c] = dp[c - w] + v
    return dp[W]


def knapsack_unbounded(items, W):
    dp = [0] * (W + 1)
    for w, v in items:
        for c in range(w, W + 1):              # UP: item reusable
            if dp[c - w] + v > dp[c]:
                dp[c] = dp[c - w] + v
    return dp[W]


def coin_ways_combinations(coins, S):
    """{1,2} and {2,1} counted ONCE (CSES 1636)"""
    dp = [1] + [0] * S
    for c in coins:                            # coins OUTSIDE
        for s in range(c, S + 1):
            dp[s] = (dp[s] + dp[s - c]) % MOD
    return dp[S]


def coin_ways_ordered(coins, S):
    """{1,2} and {2,1} counted TWICE (CSES 1635)"""
    dp = [1] + [0] * S
    for s in range(1, S + 1):                  # sum OUTSIDE
        t = 0
        for c in coins:
            if c <= s:
                t += dp[s - c]
        dp[s] = t % MOD
    return dp[S]


def min_coins(coins, S):
    dp = [0] + [INF] * S
    for s in range(1, S + 1):
        for c in coins:
            if c <= s and dp[s - c] + 1 < dp[s]:
                dp[s] = dp[s - c] + 1
    return dp[S] if dp[S] < INF else -1


def lis_length(a, strict=True):
    """tails[k] = smallest possible last value of an increasing subsequence of
length k+1.
    non-decreasing: use bisect_right."""
    tails = []
    for x in a:
        i = (bisect_left if strict else bisect_right)(tails, x)
        if i == len(tails):
            tails.append(x)
        else:
            tails[i] = x
    return len(tails)


def lcs(s, t):
    n, m = len(s), len(t)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if s[i - 1] == t[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    # reconstruct
    out, i, j = [], n, m
    while i and j:
        if s[i - 1] == t[j - 1]:
            out.append(s[i - 1]); i -= 1; j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return dp[n][m], ''.join(reversed(out))


def edit_distance(s, t):
    """insert / delete / replace, each cost 1 (CSES 1639)"""
    n, m = len(s), len(t)
    prev = list(range(m + 1))
    for i in range(1, n + 1):
        cur = [i] + [0] * m
        for j in range(1, m + 1):
            cur[j] = min(prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + (s[i - 1] !=
t[j - 1]))
        prev = cur
    return prev[m]


def grid_paths(grid):
    """count right/down paths avoiding '#' (CSES 1638)"""
    n, m = len(grid), len(grid[0])
    dp = [[0] * m for _ in range(n)]
    for r in range(n):
        for c in range(m):
            if grid[r][c] == '#':
                continue
            if r == 0 and c == 0:
                dp[r][c] = 1
                continue
            dp[r][c] = ((dp[r - 1][c] if r else 0) + (dp[r][c - 1] if c else 0)) %
MOD
    return dp[n - 1][m - 1]


def tsp(dist):
    """bitmask DP: min cost path starting at 0 visiting all nodes. O(2^n n^2), n
<= ~16 in PyPy.
    dp[mask][v] = min cost to visit exactly `mask`, ending at v."""
    n = len(dist)
    full = 1 << n
    dp = [[INF] * n for _ in range(full)]
    dp[1][0] = 0
    for mask in range(full):
        row = dp[mask]
        for v in range(n):
            cur = row[v]
            if cur == INF or not (mask >> v) & 1:
                continue
            for u in range(n):
                if not (mask >> u) & 1:
                    nm = mask | (1 << u)
                    if cur + dist[v][u] < dp[nm][u]:
                        dp[nm][u] = cur + dist[v][u]
    return min(dp[full - 1])


def interval_merge_cost(a):
```

```
    """interval DP: merge adjacent piles, cost = sum of the two merged. dp[l][r]
over lengths."""
    n = len(a)
    P = [0]
    for x in a:
        P.append(P[-1] + x)
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n + 1):
        for l in range(n - length + 1):
            r = l + length - 1
            dp[l][r] = min(dp[l][k] + dp[k + 1][r] for k in range(l, r)) + P[r +
1] - P[l]
    return dp[0][n - 1]


def digit_count(N):
    """count x in [0, N] whose digits satisfy a property, via digit DP.
    Example property here: no two adjacent equal digits (CSES 2220 style, leading
zeros skipped).
    State: (pos, last digit, tight, started)."""
    D = list(map(int, str(N)))

    @lru_cache(None)
    def go(pos, last, tight, started):
        if pos == len(D):
            return 1
        lim = D[pos] if tight else 9
        total = 0
        for d in range(lim + 1):
            if started and d == last:
                continue
            ns = started or d != 0
            total += go(pos + 1, d if ns else -1, tight and d == lim, ns)
        return total
    return go(0, -1, True, False)


def max_subarray(a):
    """Kadane: best non-empty contiguous sum"""
    best = cur = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best


def tree_max_independent(n, edges):
    """tree DP: max nodes chosen with no two adjacent. Iterative (no recursion
limit)."""
    adj = [[] for _ in range(n)]
    for u, v in edges:
        adj[u].append(v); adj[v].append(u)
    order, parent, stack = [], [-1] * n, [0]
    seen = [False] * n
    seen[0] = True
    while stack:
        v = stack.pop()
        order.append(v)
        for u in adj[v]:
            if not seen[u]:
                seen[u] = True
                parent[u] = v
                stack.append(u)
    take, skip = [1] * n, [0] * n
    for v in reversed(order):                  # children before parents
        p = parent[v]
        if p >= 0:
            take[p] += skip[v]
            skip[p] += max(take[v], skip[v])
    return max(take[0], skip[0])
```

10. Strings (strings.py)

```
"""Strings page. Self-test: python3 strings.py

SPOT IT
  find pattern in text, count occurrences  -> KMP prefix function or Z  (or s.find
/ s.count
                                                for small inputs: .count is NON-
overlapping!)
  period / smallest repeating unit         -> prefix function: p = n - pi[-1], if
n % p == 0
  compare many substrings fast             -> rolling hash (two mods)
  longest palindromic substring            -> Manacher O(n)  (or expand-around-
center O(n^2))
  many words, prefix queries               -> trie (dict of dicts)
  sorted suffixes / distinct substrings    -> suffix array (O(n log^2 n) below)
Python: s[::-1], ''.join(list), ord(c)-97, chr(97+i), str.startswith,
collections.Counter(s)
Build strings with a list + ''.join, never s += c in a loop.
"""


def prefix_function(s):
    """pi[i] = length of longest proper prefix of s[:i+1] that is also its
suffix"""
    n = len(s)
    pi = [0] * n
    for i in range(1, n):
        k = pi[i - 1]
        while k and s[i] != s[k]:
            k = pi[k - 1]
        if s[i] == s[k]:
            k += 1
        pi[i] = k
    return pi


def kmp_find(text, pat):
    """all start indices of pat in text (overlapping allowed)"""
    if not pat:
        return list(range(len(text) + 1))
    s = pat + '\x00' + text
    pi = prefix_function(s)
    m = len(pat)
    return [i - 2 * m for i in range(len(s)) if pi[i] == m]


def z_function(s):
    """z[i] = length of longest common prefix of s and s[i:]  (z[0] = n)"""
    n = len(s)
    z = [0] * n
    if n:
        z[0] = n
    l = r = 0
    for i in range(1, n):
        if i < r:
            z[i] = min(r - i, z[i - l])
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1
```

```
        if i + z[i] > r:
            l, r = i, i + z[i]
    return z


class Hash:
    """rolling hash with two mods. h.get(l, r) = hash of s[l:r] (half-open).
    Equal substrings <=> equal hashes (with overwhelming probability)."""
    M1, M2, B = 10**9 + 7, 998244353, 131

    def __init__(self, s):
        n = len(s)
        M1, M2, B = self.M1, self.M2, self.B
        self.h1 = h1 = [0] * (n + 1)
        self.h2 = h2 = [0] * (n + 1)
        self.p1 = p1 = [1] * (n + 1)
        self.p2 = p2 = [1] * (n + 1)
        for i, ch in enumerate(s):
            c = ord(ch)
            h1[i + 1] = (h1[i] * B + c) % M1
            h2[i + 1] = (h2[i] * B + c) % M2
            p1[i + 1] = p1[i] * B % M1
            p2[i + 1] = p2[i] * B % M2

    def get(self, l, r):
        return ((self.h1[r] - self.h1[l] * self.p1[r - l]) % self.M1,
                (self.h2[r] - self.h2[l] * self.p2[r - l]) % self.M2)


def manacher(s):
    """returns (start, length) of the longest palindromic substring, O(n)"""
    t = '^#' + '#'.join(s) + '#$'
    n = len(t)
    P = [0] * n
    c = r = 0
    for i in range(1, n - 1):
        if i < r:
            P[i] = min(r - i, P[2 * c - i])
        while t[i + P[i] + 1] == t[i - P[i] - 1]:
            P[i] += 1
        if i + P[i] > r:
            c, r = i, i + P[i]
    length, center = max((v, i) for i, v in enumerate(P))
    return (center - length) // 2, length


class Trie:
    def __init__(self):
        self.root = {}

    def insert(self, w):
        node = self.root
        for ch in w:
            node = node.setdefault(ch, {})
            node['#'] = node.get('#', 0) + 1      # words passing through here
        node['$'] = node.get('$', 0) + 1          # words ending here

    def count_prefix(self, p):
        node = self.root
        for ch in p:
            if ch not in node:
                return 0
            node = node[ch]
        return node.get('#', 0)                   # number of words starting with
p (p non-empty)
```

```
    def contains(self, w):
        node = self.root
        for ch in w:
            if ch not in node:
                return False
            node = node[ch]
        return node.get('$', 0) > 0


def suffix_array(s):
    """O(n log^2 n) prefix doubling. sa[i] = start of i-th smallest suffix."""
    n = len(s)
    rank = [ord(c) for c in s]
    sa = list(range(n))
    k = 1
    while True:
        key = lambda i: (rank[i], rank[i + k] if i + k < n else -1)
        sa.sort(key=key)
        new = [0] * n
        for j in range(1, n):
            new[sa[j]] = new[sa[j - 1]] + (key(sa[j]) != key(sa[j - 1]))
        rank = new
        if n == 0 or rank[sa[-1]] == n - 1:
            return sa
        k <<= 1


def lcp_array(s, sa):
    """Kasai: lcp[i] = common prefix of suffixes sa[i] and sa[i+1].
    distinct substrings = n(n+1)/2 - sum(lcp)"""
    n = len(s)
    rank = [0] * n
    for i, p in enumerate(sa):
        rank[p] = i
    lcp, h = [0] * max(n - 1, 0), 0
    for i in range(n):
        if rank[i] + 1 < n:
            j = sa[rank[i] + 1]
            while i + h < n and j + h < n and s[i + h] == s[j + h]:
                h += 1
            lcp[rank[i]] = h
            if h:
                h -= 1
        else:
            h = 0
    return lcp


def min_rotation(s):
    """index of lexicographically smallest rotation (Booth-like, O(n))"""
    s2, n = s + s, len(s)
    i, j, k = 0, 1, 0
    while i < n and j < n and k < n:
        a, b = s2[i + k], s2[j + k]
        if a == b:
            k += 1
            continue
        if a > b:
            i += k + 1
        else:
            j += k + 1
        if i == j:
            j += 1
```

```
        k = 0
    return min(i, j)
```